ESTADO DEL BACKEND — B-Side Club (TPO Plataforma web para pequeños comercios)

Stack: Node.js + Express + Prisma (PostgreSQL/Supabase) + JWT. JavaScript plano, sin TypeScript.
Base URL local: http://localhost:3000/api
Numeración según las funciones principales que pide el enunciado del TPO.

---

1. SITIO PUBLICO (VISITANTE, SIN LOGIN)

1.1 Informacion general del comercio

```
GET /api/store-info
Nombre, descripcion, direccion, telefono, mail, instagram, whatsapp,
horario (JSON).
```

1.2 Visualizar productos y/o servicios ofrecidos

```
GET /api/productos
```

```
GET /api/productos/:id  (detalle con categoria e imagenes, ordenadas
por el campo "orden" de cada imagen)
Solo muestra productos con esActivo=true — nunca los dados de baja.
```

1.3 Buscar publicaciones

```
GET /api/productos?q=texto
Busca por nombre y descripcion, sin distinguir mayusculas/minusculas.
```

1.4 Filtrar publicaciones por categoria

```
GET /api/productos?categoriaId=N
(tambien soporta ?destacado=true para la seccion de destacados/home)
```

1.5 Consultar informacion de contacto

Incluida en GET /api/store-info (telefono, mail, instagram, whatsapp).

1.6 Enviar consultas al comercio

```
POST /api/consultas — nombre, email (validado), asunto, mensaje
obligatorios; telefono y productoId opcionales. Queda guardada con
estado="pendiente".
```

Paginacion del catalogo: GET /api/productos soporta page y pageSize (default 12, maximo 60). Devuelve { productos, page, pageSize, total, totalPages }.

2. REGISTRO E INICIO DE SESION (ADMINISTRADOR)

2.1 Registrarse

```
POST /api/auth/register — crea el primer usuario (queda como
super_admin). El resto del staff se suma via invitacion (seccion 4),
no por este endpoint.
Datos minimos pedidos: nombre, apellido, mail, telefono, contraseña.
Mail duplicado rechazado (constraint unico en la base).
```

2.2 Iniciar sesion

```
POST /api/auth/login — devuelve { token, user }. El JWT vence a
los 7 dias (JWT_EXPIRES_IN en .env).
```

2.3 Cerrar sesion

```
POST /api/auth/logout (protegida) — el token es stateless, lo
descarta el frontend; este endpoint solo confirma la accion.
```

2.4 Recuperar contraseña

```
POST /api/auth/forgot-password — genera un token valido 30 minutos
y lo manda por mail (o lo loguea por consola si no hay SMTP
configurado). Respuesta identica exista o no el mail, para no
filtrar que usuarios estan registrados.
```

```
POST /api/auth/reset-password — valida token + contraseña nueva
y actualiza el hash.
```

2.5 Modificar sus datos personales

```
GET /api/auth/me y PUT /api/auth/me (protegidas).
```

Validaciones aplicadas: formato de mail (regex) y contraseña minima de 6 caracteres, tanto en registro como en reset y aceptar invitacion.

3. ADMINISTRACION DE INFORMACION INSTITUCIONAL

```
PUT /api/store-info (protegida: cualquier rol de staff)
```

Upsert sobre la fila unica de la tabla StoreInfo (id=1): nombre, descripcion, direccion, telefono, redes sociales (instagram/whatsapp) y horarios de atencion.

4. ADMINISTRACION DE PRODUCTOS Y SERVICIOS

4.1 Crear publicaciones

```
POST /api/productos (protegida) — valida campos requeridos y que
la categoria exista.
```

4.2 Modificar publicaciones

```
PUT /api/productos/:id (protegida) — edita cualquier campo.
```

4.3 Eliminar publicaciones

```
DELETE /api/productos/:id (protegida) — borra el producto y sus
imagenes en cascada.
```

4.4 Activar o desactivar publicaciones

Se logra con PUT /api/productos/:id mandando { esActivo: false }. No hay un endpoint PATCH separado para esto — es la misma via que editar cualquier otro campo.

4.5 Imagenes de cada publicacion (multiples, con portada)

```
POST /api/productos/:id/imagenes (protegida) — sube hasta 5
imagenes reales con multer. El orden sigue la secuencia existente
del producto (no pisa la portada al agregar una tanda nueva).
```

```
PATCH /api/imagenes/:id (protegida) — cambia el campo "orden" de
una imagen ya subida, para reordenar cual queda como portada.
```

```
DELETE /api/imagenes/:id (protegida) — borra el registro y el
archivo fisico en /uploads.
```

4.6 Productos precargados para demostrar busqueda/filtros/navegacion

Seed con 22 productos (el minimo que pide la consigna es 20), repartidos en las 4 categorias.

Cada producto guarda: nombre, categoria, imagenes, descripcion, precio (nullable = "a consultar"), y estado de disponibilidad (enum: disponible, ultima_unidad, reservado, a_consultar, vendido) ademas de esActivo y destacado.

5. CATEGORIAS

```
GET    /api/categorias        (publica) Lista todas.
```

```
GET    /api/categorias/:id    (publica) Detalle.
```

```
POST   /api/categorias        (protegida) Crear.
```

```
PUT    /api/categorias/:id    (protegida) Modificar.
```

```
DELETE /api/categorias/:id    (protegida) Eliminar — rechaza con 409 si
hay productos usando esa categoria.
```

Cada producto esta asociado a una categoria por FK obligatoria.

6. FORMULARIO DE CONTACTO

```
POST /api/consultas (publica) — mismos datos minimos que pide la
```

consigna: nombre, correo electronico, telefono (opcional), asunto, mensaje. Se valida el formato del mail. Queda guardada en la base con canal="formulario" y estado="pendiente".

7. GESTION DE CONSULTAS (PANEL ADMIN)

```
GET    /api/consultas          (protegida) Lista, filtro opcional por
estado.
```

```
PATCH  /api/consultas/:id/estado (protegida) Cambia entre pendiente /
leida / respondida (rechaza cualquier
otro valor).
```

```
DELETE /api/consultas/:id       (protegida) Elimina.
```

8. ROLES DEL SISTEMA

Visitante: todo lo de la seccion 1, sin login.

Administrador (repartido en 3 roles del sistema: super_admin, administrador, vendedor — el enunciado pide un solo rol "administrador", se extendio a 3 para reflejar mejor un equipo real de local):

- Gestionar perfil: seccion 2.5.
- Gestionar productos/servicios: seccion 4 (los 3 roles).
- Gestionar categorias: seccion 5 (los 3 roles).
- Gestionar consultas: seccion 7 (los 3 roles).
- Administrar informacion del comercio: seccion 3 (los 3 roles).

Reglas de permisos especiales, resueltas a mano porque dependen del usuario DESTINO y no solo de quien hace el pedido:

- Invitar usuarios nuevos: solo super_admin y administrador.
- Nadie puede editar ni eliminar al super_admin.
- Nadie puede asignarse (ni asignarle a otro) el rol super_admin.
- Solo el super_admin puede eliminar a un administrador.

Invitaciones y gestion de usuarios (como se suma el resto del equipo):

```
POST   /api/invitacion            Invita por mail con un rol. Token
valido 7 dias.
```

```
GET    /api/invitacion            Lista invitaciones y su estado.
```

```
DELETE /api/invitacion/:id        Revoca una invitacion pendiente.
```

```
POST   /api/invitacion/aceptar    (publica) Crea la cuenta con el rol
de la invitacion.
```

```
GET    /api/usuarios              Lista el staff.
```

```
PUT    /api/usuarios/:id          Edita un usuario (no al super_admin).
```

```
DELETE /api/usuarios/:id          Elimina un usuario (con las reglas
de arriba).
```

9. REQUERIMIENTOS TECNICOS OBLIGATORIOS (LADO BACKEND)

- Consumo de API REST: toda la app se sirve como API JSON en /api/*.
- Persistencia de datos: PostgreSQL (Supabase) via Prisma.
- Manejo de errores: middleware centralizado (errorHandler), respuestas JSON prolijas con status code correcto, nunca tira abajo el server.
- Validaciones en backend: formato de mail, longitud de contraseña, campos requeridos, estados validos, categoria existente, categoria con productos asociados no se puede borrar.
- Autenticacion y autorizacion: JWT + roles (requireRol) + reglas por usuario destino donde corresponde.
- Separacion frontend/backend: este repo es solo la API; el frontend va aparte (React + Vite, todavia no empezado).

Servidor base: Express, cors, morgan (logs), express.json(),

```
GET /api/health, archivos subidos servidos como estaticos en /uploads.
```

10. FUNCIONALIDADES ADICIONALES (PUNTAJE EXTRA)

- Dashboard con metricas: GET /api/dashboard/metricas — devuelve consultasPendientes, productosActivos, productosInactivos, productosSinStock, invitacionesPendientes, totalUsuarios.
- Gestion de multiples imagenes por publicacion: hecho (seccion 4.5), incluye reordenar portada.
- Sistema de promociones/novedades: tabla Promocion migrada y sincronizada en la base real. CRUD completo: GET/POST /api/promociones, GET/PUT/DELETE /api/promociones/:id. Sin datos de ejemplo cargados en el seed todavia.
- Publicacion destacada de productos: campo "destacado" + filtro ?destacado=true, usado para la seccion de destacados de la home.
- Envio de correos electronicos desde el formulario de contacto: NO implementado — el formulario de contacto guarda la consulta en la base (seccion 6), pero no dispara un mail de notificacion al comercio. El sistema de mails (nodemailer) que existe hoy se usa solo para invitaciones y recuperar contraseña.
- Buscador avanzado: parcial — hoy es busqueda por texto + filtro de categoria + paginacion (seccion 1.3/1.4). No tiene ordenamiento por precio/fecha ni filtros combinados mas finos.

PENDIENTE / OPCIONAL (NO BLOQUEA EL TPO)

- Envio real de mails: el codigo esta listo (nodemailer), pero sin credenciales SMTP en .env cae a loguear por consola. Hace falta cargar SMTP_HOST/USER/PASS reales para que cuente como "envio real" en el puntaje extra.
- Cargar promociones de ejemplo en el seed.
- Sacar prisma/migrations del .gitignore para que el equipo comparta un solo historial de migraciones (hoy cada uno usa el suyo local; los cambios de schema se aplican con "npx prisma db push").
- Frontend: no existe todavia, hay que construirlo (React + Vite), usando demo/demo.html como referencia visual.